

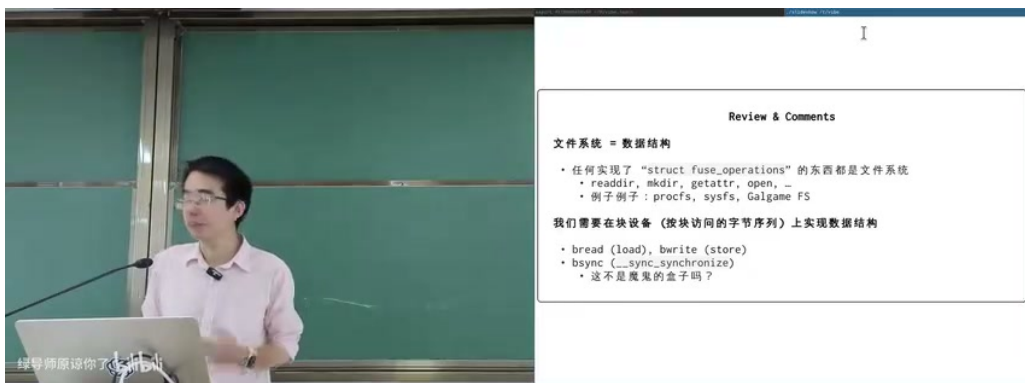
课程笔记

26 - 文件系统实现和崩溃一致性

2026 南京大学操作系统原理

lecture-to-notes

2026 年



视频作者/频道：南京大学操作系统原理（蒋炎岩）

发布日期：2026

视频时长：01:27:02

视频链接：未填写（本地课程视频）

目录

1	从块设备到文件系统	3
1.1	块设备模型	3
1.2	虚拟块设备和 LVM	4
1.3	缓存、局部性和读写放大	4
1.4	本章小结	5
2	把文件系统看作磁盘上的数据结构	5
2.1	Super Block: 整个数据结构的入口	5
2.2	FAT: 极简的链表式文件分配	6
2.3	把磁盘镜像当结构体解析	7
2.4	目录项: 名字到数据的桥	7
2.5	UNIX inode: 元数据、链接和多级索引	8
2.6	本章小结	9
3	崩溃一致性: 持久化数据结构的真正难题	9
3.1	一致状态到一致状态	10
3.2	多块写入的任意子集	10
3.3	fsck: 事后修补的时代	11
3.4	本章小结	12
4	日志: 用追加记录拯救任意数据结构	12
4.1	从真实结构到操作序列	12
4.2	文件系统的事务日志	13
4.3	LSM tree 的旁支启发	14
4.4	JBD、checksum 和 metadata journaling	14
4.5	本章小结	15
5	应用程序也要面对崩溃一致性	15
5.1	错误的保存文件方式	15
5.2	sync 系列系统调用	16
5.3	安全保存: 临时文件、同步、原子 rename	17
5.4	从论文到可复现实验	17
5.5	本章小结	18
6	总结与延伸	18
6.1	机制总结	18
6.2	常见误区	19
6.3	给实验和工程的建议	19

6.4	拓展阅读	19
6.5	最终 takeaways	19

1 从块设备到文件系统

上一讲已经把文件系统的外部形态讲清楚了：它向用户暴露目录树，支持遍历目录、创建/删除目录、读取属性、打开文件、读写文件等操作。无论是 `procfs`、`sysfs`、`FUSE` 实现的用户态文件系统，还是把游戏状态伪装成目录树的玩具例子，只要对外满足这组接口，它都可以被看成文件系统。本讲进入实现层：如何在块设备上构造这个目录树，并且在系统崩溃时仍保持持久化数据结构的一致性。

1.1 块设备模型

讲义使用一个简化但非常有用的块设备抽象：

块设备的三个基本动作

块设备可以被看成一个 `block array`。文件系统实现者主要依赖三个动作：读取一个块、写入一个块、等待此前提提交的写入落盘。最后一个动作可以类比并发程序中的 `barrier`：它建立“之前的写入已经完成”的顺序关系。

这个类比很重要。内存是字节序列，程序用 `load/store` 实现链表、树、哈希表；块设备是块序列，文件系统用 `bread/bwrite/sync` 实现目录树、索引、位图、日志。区别在于，内存数据结构崩溃后可以丢弃，而文件系统的数据结构在掉电后仍必须能解释。

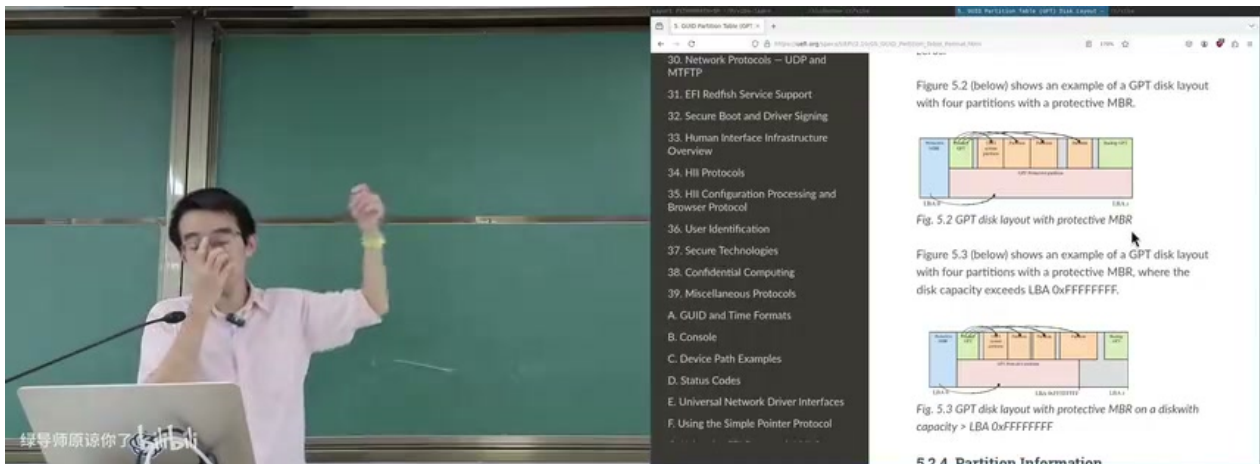


图 1：块设备本身也有数据结构：GPT 分区表描述了磁盘上的分区布局，并在磁盘头尾保存关键信息。¹

真实设备比这个模型复杂。一个磁盘或 SSD 的前部可能是 GUID Partition Table，固件和操作系统启动时要解析它，才能知道系统分区、数据分区和可启动分区的位置。一个没有有效分区表的设备仍然是块设备，但系统无法进一步识别其中有哪些文件系统。

¹视频画面时间区间：00:04:35–00:04:40。

1.2 虚拟块设备和 LVM

块设备不一定直接对应物理硬盘。Linux 允许在块设备上再构造块设备：loopback device 可以把普通文件变成块设备，LVM 可以把多个物理卷组合成 volume group，再切出多个 logical volume；也可以把一个大设备划分成多个分区。RAID、device mapper、NVMe namespace 都可以放在这个视角中理解。

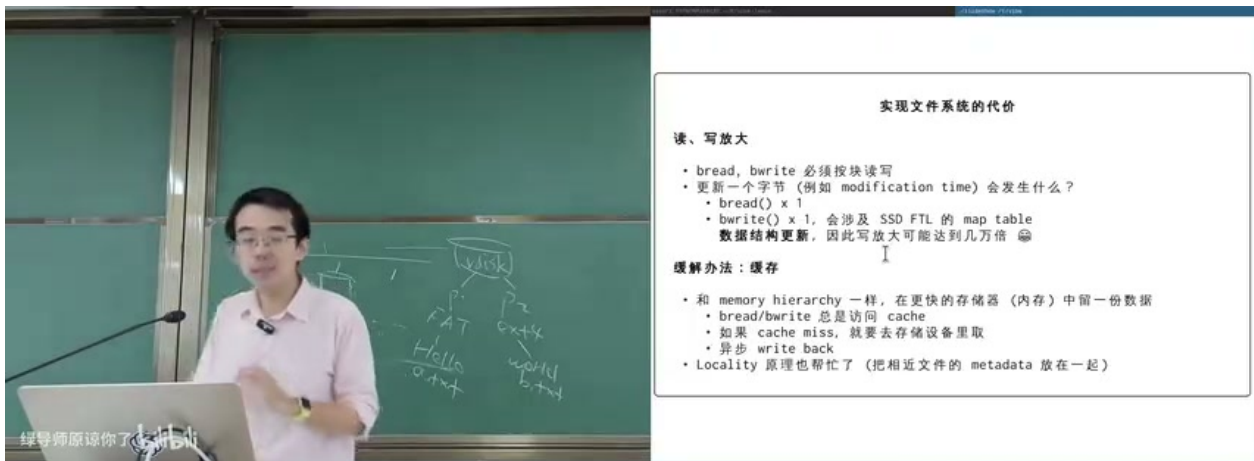
一层一层的虚拟化

块设备抽象的威力在于它只要求下层看起来能读块、写块、等待同步。因此，一个镜像文件可以经 loop device 变成块设备，多个 loop device 可以经 LVM 变成一个逻辑卷，逻辑卷上再建立 GPT、FAT32、ext4。文件系统不必知道最底层到底是 SD 卡、NVMe、RAID，还是普通文件。

课程中的 LVM demo 用多个小镜像文件创建 loopback device，再创建物理卷、卷组、逻辑卷和分区，最后分别格式化成 FAT 与 ext4。用 `strace` 观察这些工具，会发现大量控制面 API 都通过 `ioctl` 暴露：`/dev/loop-control` 用来分配 loop device，`/dev/mapper/control` 用来创建 device mapper 设备。这也解释了兼容 Linux 为什么困难：POSIX 中的 `open/read/write` 只是很小的一部分，许多内核子系统还通过大量专用 `ioctl` 接口暴露。

1.3 缓存、局部性和读写放大

文件系统读写的是块，但应用常常只关心一个字节、一个文件长度或一个时间戳。这带来读放大和写放大：为了更新文件内容的一个字节，系统可能要读出整个块、修改一部分、写回整个块，还要更新 inode 或目录项中的修改时间。SSD 内部还有 Flash Translation Layer，它把逻辑块映射到物理页，更新映射表又可能触发 copy-on-write，因此小写入会进一步被放大。



实现文件系统的代价

读、写放大

- `bread`, `bwrite` 必须按块读写
- 更新一个字节（例如 `modification time`）会发生什么？
 - `bread()` x 1
 - `bwrite()` x 1, 会涉及 SSD FTL 的 `map table` 数据结构更新，因此写放大可能达到几万倍

缓解办法：缓存

- 和 `memory hierarchy` 一样，在更快的存储器（内存）中留一份数据
 - `bread/bwrite` 总是访问 `cache`
 - 如果 `cache miss`，就要去存储设备里取
 - 异步 `write back`
 - `Locality` 原理也帮忙了（把相近文件的 `metadata` 放在一起）

图 2：文件系统在块设备之上维护缓存：`bread` 类似 `load`，`bwrite` 类似 `store`，但写入可能先停留在内存 `cache` 中。²

²视频画面时间区间：00:13:05–00:13:10。

因此，文件系统必须像内存层次结构一样利用局部性。经常一起访问的数据应该尽量放在相邻块或同一块中；元数据缓存能减少频繁 `stat`、目录遍历、路径解析带来的 I/O。Linux 历史上称这一层为 `buffer cache`；从概念上看，它就是文件系统缓存。

1.4 本章小结

文件系统的下层不是神秘接口，而是一个可读、可写、可同步的块数组。真实系统中，块数组会经过分区表、`loopback`、`LVM`、`device mapper`、`RAID` 等层层虚拟化；但一旦抽象稳定，上层文件系统就可以把它当作持久化数据结构的存储介质。性能问题来自块粒度和实际访问粒度的不匹配，因此缓存与局部性是文件系统实现的第一原则。

2 把文件系统看作磁盘上的数据结构

文件系统要实现的是目录、文件、链接、权限、时间戳和文件内容组成的数据结构。把这件事放回数据结构课看，问题可以拆成三类：入口在哪里，元数据怎么存，文件内容的块索引怎么存。

2.1 Super Block：整个数据结构的入口

任何持久化数据结构都需要一个 `root`。ELF 有 `header`，BMP/PPM 有格式头，GPT 有分区表头；文件系统也有自己的 `root`，通常叫 `super block`。它记录文件系统的 `magic number`、逻辑块大小、总块数、空闲块数、`inode` 数量、各个区域的起始位置等关键信息。

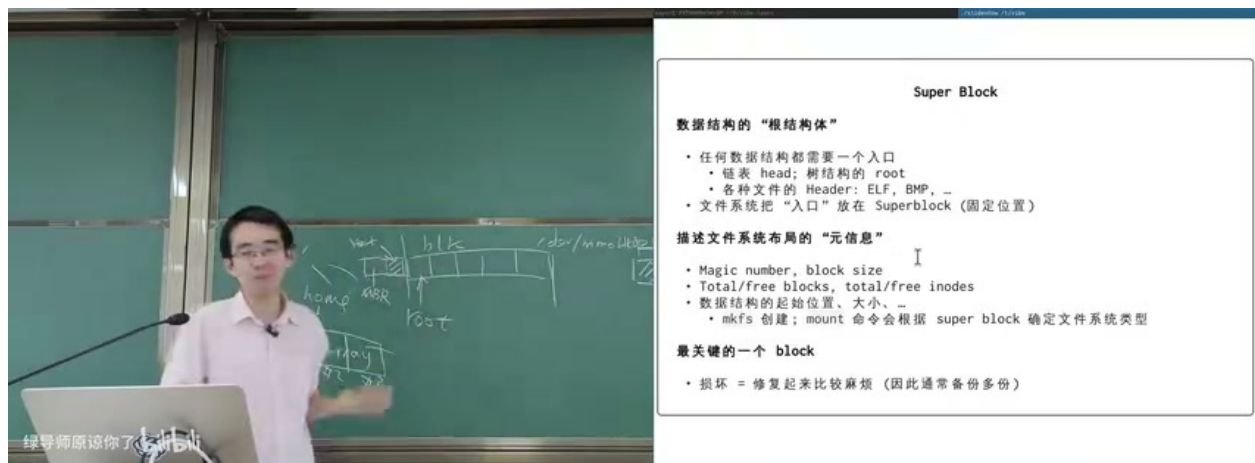


图 3: Super Block 保存文件系统的全局元数据，是 `mount`、识别格式、定位后续数据结构的入口。³

如果只能破坏一个块并造成最大麻烦，`super block` 通常是最值得攻击的目标。它不是所有数据本身，却是解释所有数据的入口；没有它，后续 `inode` 表、位图、数据区的位置都要通过恢复工具重新推断。

³视频画面时间区间：00:24:50–00:24:55。

2.2 FAT：极简的链表式文件分配

FAT 的设计背景是极小的存储设备。例如 5.25 英寸软盘单面约 160 KB，只有 320 个 512 字节扇区。在这样的介质上，复杂索引结构本身就是空间浪费。FAT 的核心思想是：把数据区划分成 cluster，然后用一个紧凑数组记录每个 cluster 的下一个 cluster。

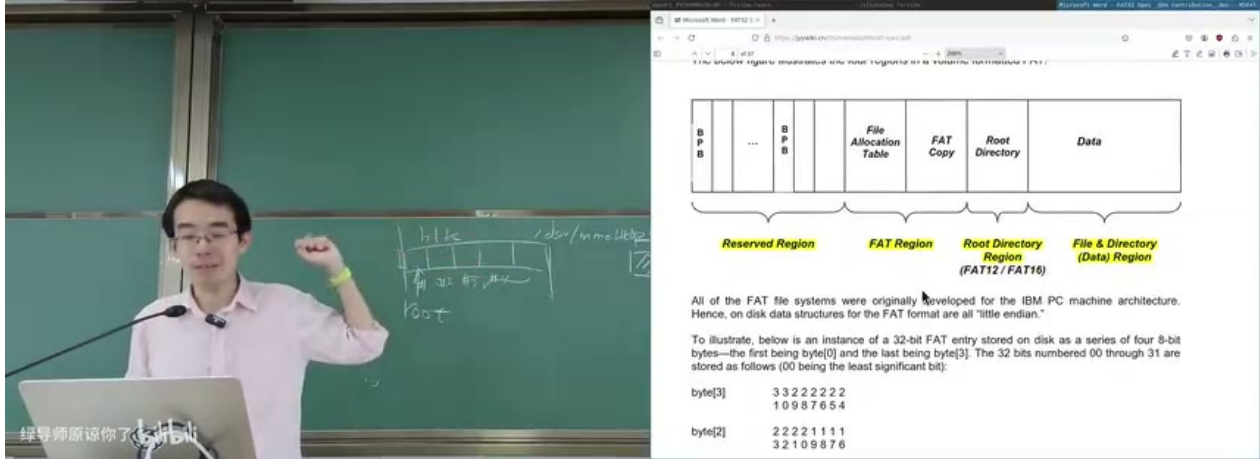


图 4: FAT 规范中的 BPB/布局字段展示了一个文件系统如何把 boot sector、FAT、root directory 和 data region 串起来。⁴

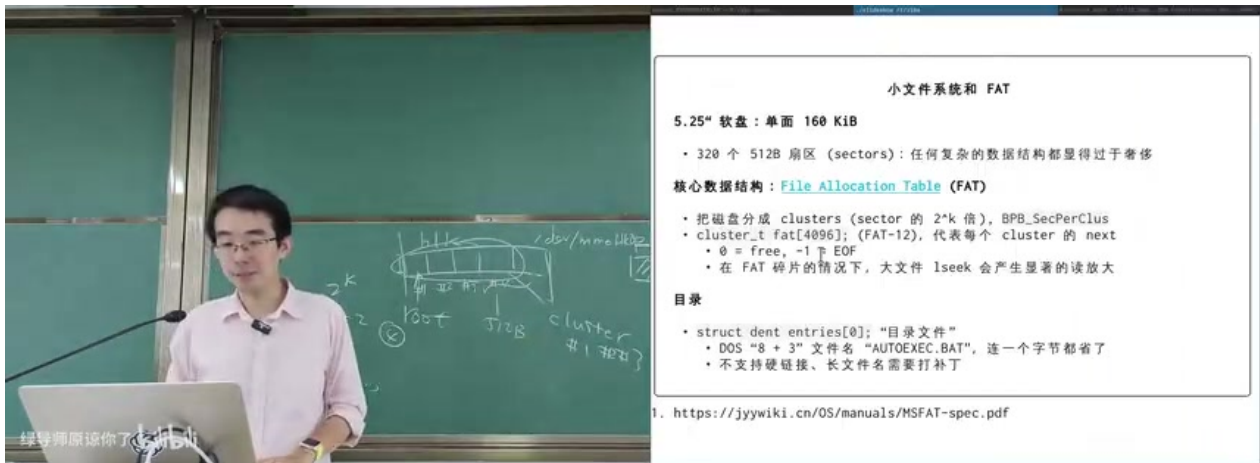


图 5: FAT 的本质是 next 数组：每个 cluster entry 指向文件的下一块，0 表示空闲，特殊值表示文件结束。⁵

对于一个 FAT 文件，目录项保存第一个 cluster 和文件大小；读文件时沿着 FAT 表中的 next 指针依次遍历。FAT12 中一个 entry 只有 12 bit，两个 entry 可以打包进 3 个字节，这体现了早期文件系统对空间的极致节约。

⁴视频画面时间区间：00:27:55–00:28:00。

⁵视频画面时间区间：00:29:35–00:29:40。

FAT 的碎片化问题

当文件不断创建、删除、再创建时，空闲 cluster 会像内存分配一样碎片化。大文件一旦由很多不连续 cluster 组成，读取时就需要在 FAT 表中频繁跳转，甚至不断读入不同的 FAT 块，造成严重读放大。FAT 适合小设备和小文件，并不适合大规模随机更新的现代工作负载。

2.3 把磁盘镜像当结构体解析

课程 demo 中用 mmap 把一个 FAT32 镜像映射到进程地址空间，然后把 super block 直接解释成 C 结构体指针。接下来通过 DFS 扫描目录项，读出每个文件的 first cluster、大小、属性，再沿 FAT 表遍历文件内容。

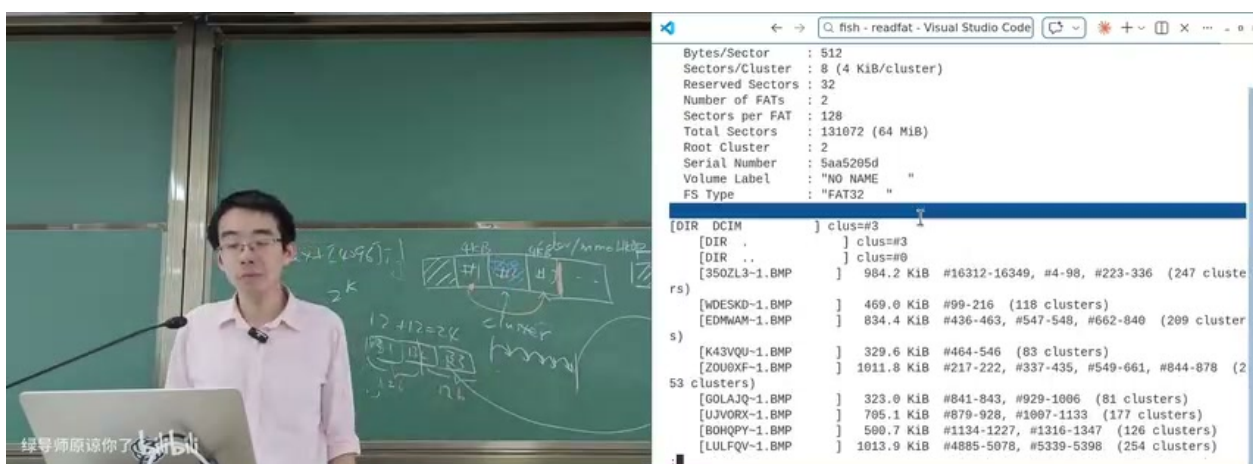


图 6: FAT32 镜像扫描工具输出了目录树、文件大小、cluster 范围和碎片分布，说明文件系统可以被普通程序按数据结构解析。⁶

这个视角对实验很关键：文件系统恢复不是魔法，而是在一段字节序列中重新识别结构体、数组、链表和目录树。快速格式化后很多数据仍可恢复，也是因为格式化常常只重建元数据入口，数据区的旧内容还在。

2.4 目录项：名字到数据的桥

早期 FAT 目录项使用 8.3 短文件名，11 字节固定宽度：8 字节文件名加 3 字节扩展名。目录本身也是一个文件，内容是目录项数组；每个目录项记录名字、属性、大小、时间戳和 first cluster。

⁶视频画面时间区间：00:36:05-00:36:10。

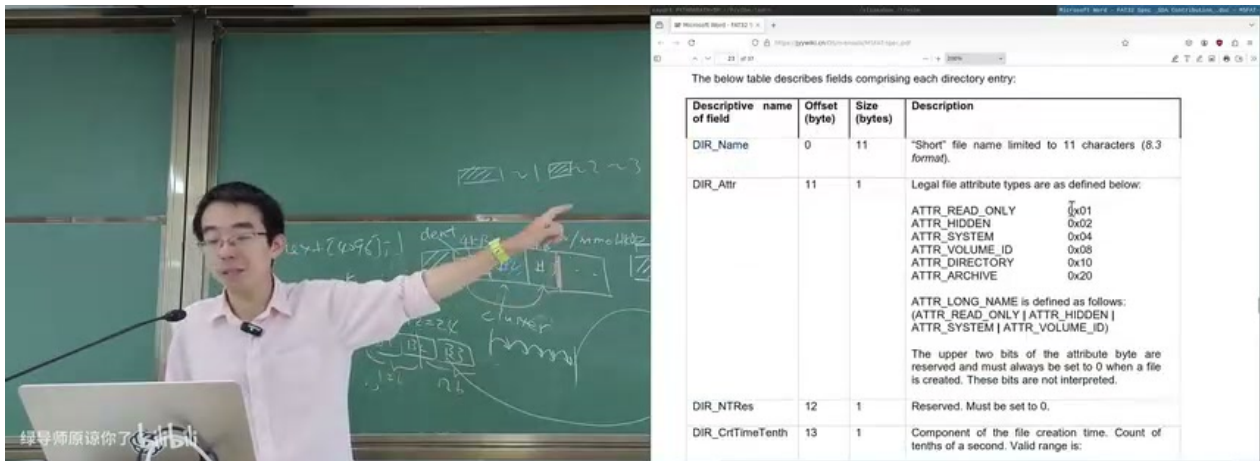


图 7: FAT directory entry 记录文件名、属性位、大小和 first cluster；目录文件就是这些定长记录的顺序表。⁷

这种设计不支持 UNIX 意义上的硬链接。目录项直接绑定文件元数据和 first cluster，更像一棵树；而 UNIX 文件系统把“名字到对象”的边和“对象本身的元数据”分离，形成更一般的图结构。

2.5 UNIX inode：元数据、链接和多级索引

UNIX 文件系统把每个文件或目录抽象为 inode。inode 保存权限、所有者、大小、时间戳、链接计数以及数据块索引；目录文件则保存从文件名到 inode 编号的映射。这样，多个目录项可以指向同一个 inode，硬链接自然成立；目录中的 . 和 .. 也使目录结构成为图，而不是单纯的树。

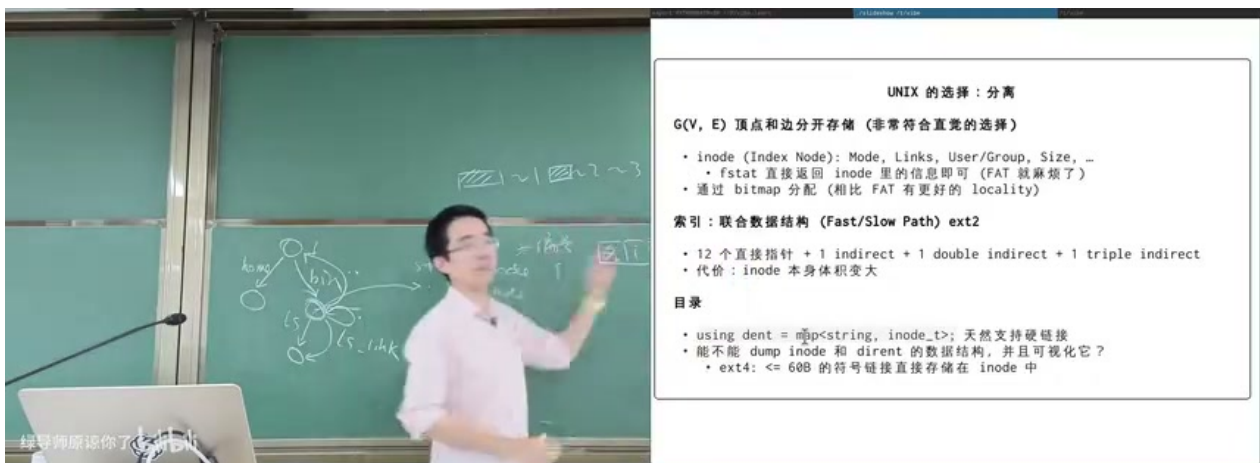


图 8: UNIX inode 用直接块、一级间接、二级间接、三级间接组合索引文件内容，同时把元数据集中存放。⁸

这个设计是典型的 workload-aware 数据结构。多数文件很小，直接块指针使读取文件

⁷视频画面时间区间：00:39:45–00:39:50。

⁸视频画面时间区间：00:44:10–00:44:15。

头部非常快；少数文件很大，间接块提供指数级扩展能力。假设块大小为 4 KB、块号为 4 字节，一个间接块可容纳 1024 个块号：

$$\text{三级间接可索引的数据量} \approx 1024^3 \times 4\text{KB}$$

符号说明：1024 是一个 4 KB 间接块能容纳的块号数量；三次方对应三级索引；4KB 是每个数据块大小。

inode 还有小技巧。短符号链接可以直接存放在 inode 中原本用于块指针的空间里，不再单独分配数据块。这是结构体空间复用的例子：当一个文件类型不需要某些字段时，这些字节可以承载别的含义。

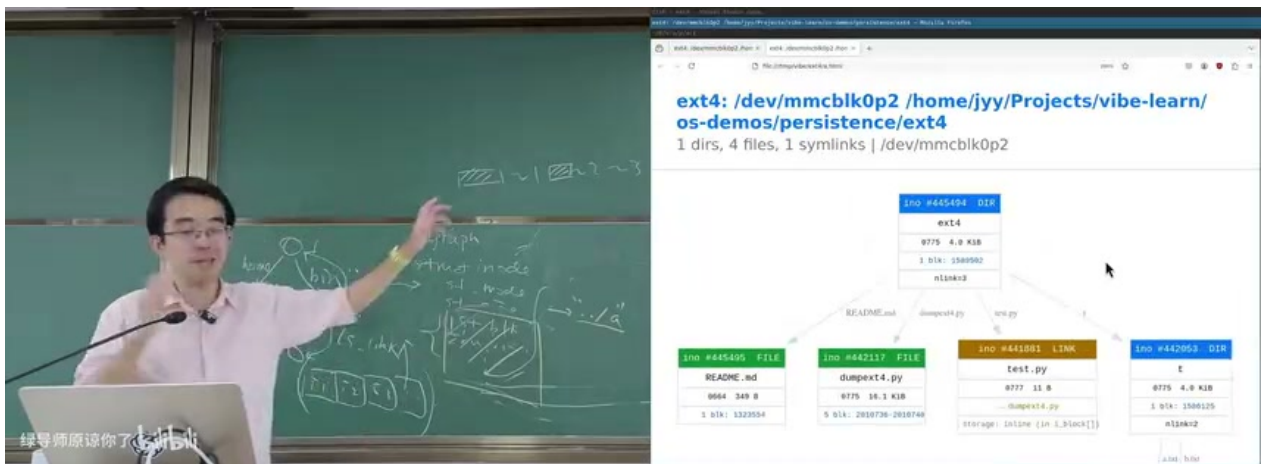


图 9: ext4 可视化工具借助 debugfs 展示 inode、link count、数据块范围和 inline symbolic link。⁹

2.6 本章小结

FAT 与 UNIX inode 展示了两种不同的文件系统数据结构哲学。FAT 用极简 next 数组换空间紧凑和实现简单，适合小介质；UNIX inode 把目录边、对象元数据、数据块索引分离，支持链接和大文件，并通过直接/间接索引匹配“小文件多、大文件少”的统计事实。文件系统的设计首先是数据结构设计，其次才是系统调用实现。

3 崩溃一致性：持久化数据结构的真正难题

如果只考虑正常运行，文件系统实现可以完全像数据结构作业：分配块、更新位图、改 inode、写目录项。但持久化介质有一个额外敌人：系统可能在任意时刻掉电。内存状态会消失，已经提交给设备的写请求可能以任意顺序、任意子集到达磁盘。

⁹视频画面时间区间：00:49:25–00:49:30。

3.1 一致状态到一致状态

Crash consistency 要求文件系统从一个 consistent state 转移到另一个 consistent state。崩溃可以让最近操作丢失，但不能让磁盘上的数据结构变成无法解释或自相矛盾的状态。

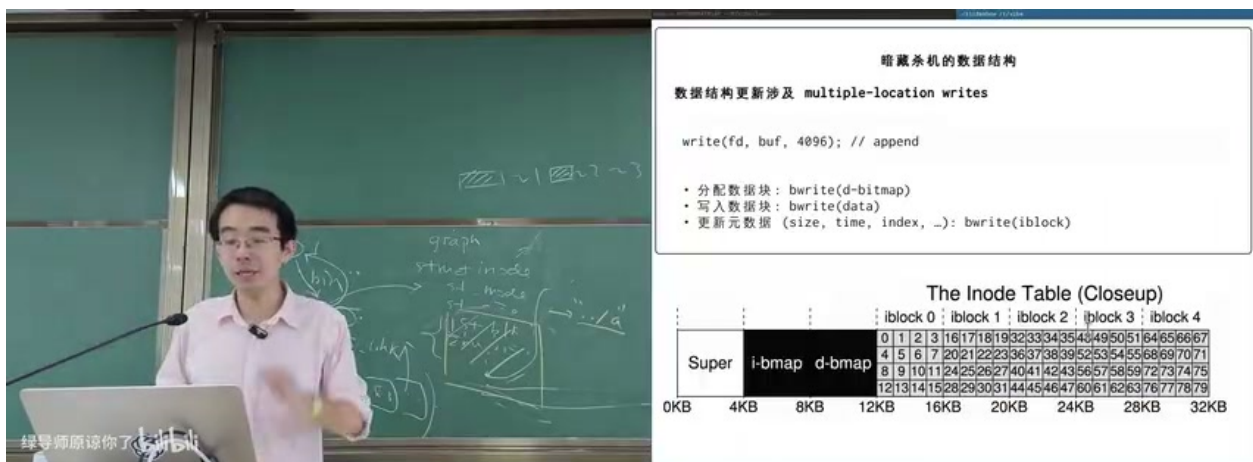


图 10: 崩溃一致性的目标：每次操作要么没有发生，要么把文件系统从一个一致状态带到另一个一致状态。¹⁰

把一个节点插入双向链表，需要更新多个指针。如果线程在中间消失，链表可能出现一边指向新节点、另一边仍指向旧节点的半更新状态。文件系统也是如此：一次看似简单的追加 4 KB，至少涉及三个写入。

```
1 write(file, buf, 4096):
2     block = balloc();           // 更新 block bitmap
3     bwrite(block, buf);        // 写入文件数据
4     inode.size += 4096;        // 更新 inode size 和 block index
5     inode.blocks[last] = block;
```

Listing 1: 追加一个块时的概念性更新

3.2 多块写入的任意子集

真实设备会重排写请求。HDD 可能按磁头位置调度，SSD 内部也有控制器、队列和 FTL。文件系统调用了三个 `bwrite`，并不意味着磁盘会按程序顺序完成这三个写入。崩溃时，三个写入的任意子集都可能已经持久化。

¹⁰ 视频画面时间区间：00:51:15–00:51:20。

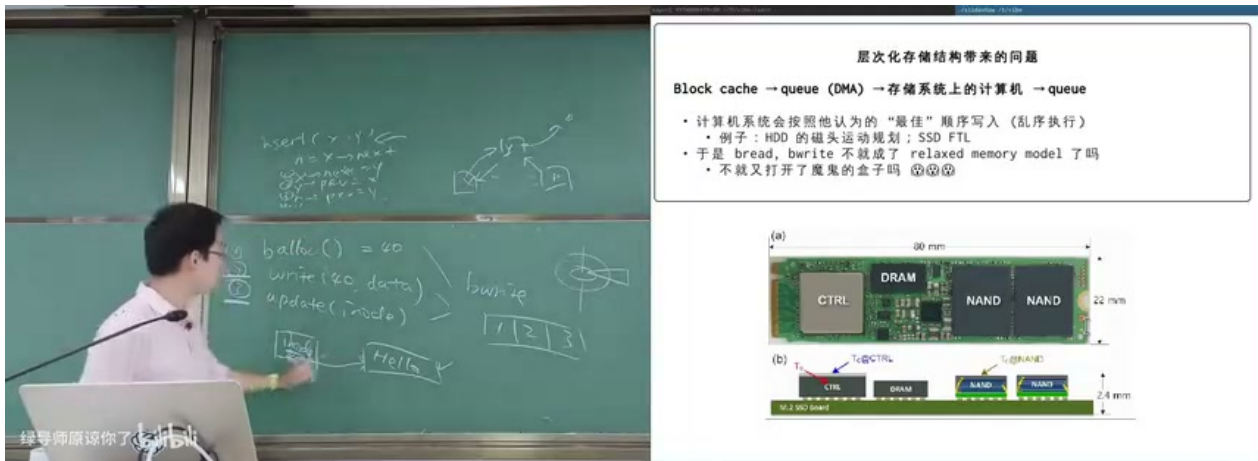


图 11: 追加一个块需要更新 bitmap、data block 和 inode；崩溃时任意子集落盘都可能产生不同后果。¹¹

例如，数据块和 inode 都写入了，但 bitmap 没有标记该块已占用。文件暂时看起来正常，未来分配器却可能把这块再次分给另一个文件，导致旧文件内容被悄悄覆盖。这种错误比立即崩溃更危险，因为它会延迟暴露。

对于 n 个独立块写入，崩溃时可能看到的落盘子集数量是：

$$2^n$$

符号说明： n 是一次高层文件系统操作涉及的块写入数量；每个写入都有“已落盘/未落盘”两种状态；实际还要考虑设备是否部分写坏一个块、写缓存是否带电保护等硬件细节。

3.3 fsck：事后修补的时代

早期系统常把一致性恢复交给 **fsck**。重启后，文件系统检查器扫描所有 inode 和目录项，重建“哪些块被引用”的集合，再与 bitmap 对照。如果 inode 引用第 40 块，而 bitmap 认为第 40 块空闲，**fsck** 可以修正 bitmap。孤儿 inode 可能被挂到 **lost+found**。

¹¹视频画面时间区间：00:56:15–00:56:20。

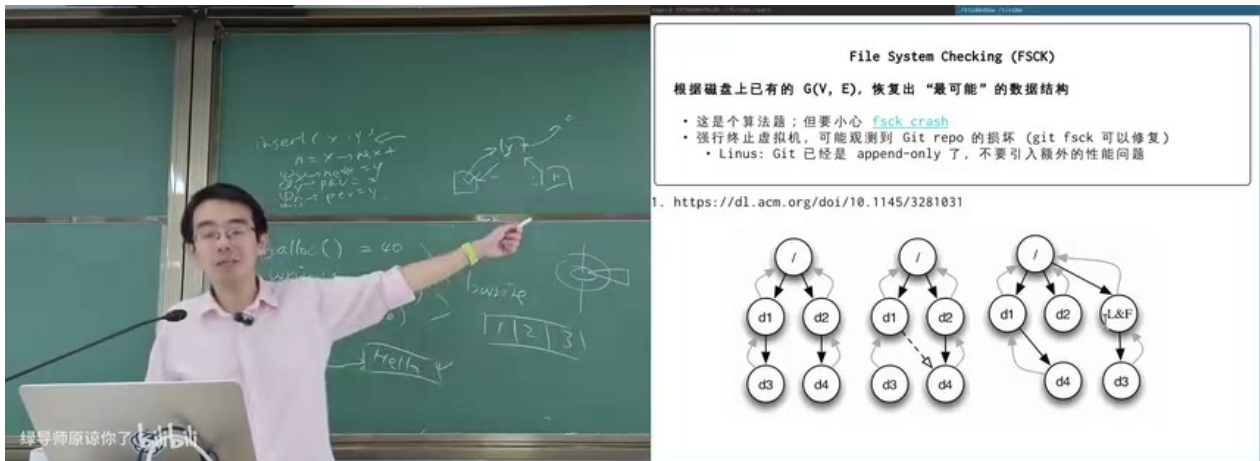


图 12: fsck 试图从损坏后的图结构中恢复一致解释; 无法归属的文件可能进入 lost+found。¹²

事后修补不是崩溃安全

fsck 可以修复某些结构性矛盾, 却常常不知道用户原本想做什么。目录移动、重命名、删除和分配交错后, 磁盘状态可能有多种合理解释。真正理想的文件系统不是“崩溃后尽量猜”, 而是“任意时刻崩溃都能按规则恢复到某个明确状态”。

3.4 本章小结

崩溃一致性的难点来自持久化写入的乱序和部分完成。内存数据结构只需在运行时被锁保护; 磁盘数据结构还要面对线程、内核、设备和电源在任意点消失。只靠事后扫描可以还债, 但无法从根本上给出强语义保证。下一步需要把“操作本身”也持久化下来。

4 日志: 用追加记录拯救任意数据结构

追加写入天然容易做到崩溃一致: 要么一条记录完整存在, 要么不存在; 如果能识别尾部不完整记录, 就可以丢弃它。日志的基本想法是把复杂的就地更新转化成可顺序追加、可重放的记录。

4.1 从真实结构到操作序列

同一个数据结构有两种表示: 一种是当前结构本身, 例如树节点及其左右指针; 另一种是从空结构开始的操作序列, 例如 insert 3、insert 6、insert 5。如果操作序列可靠保存, 即使当前结构坏了, 也可以 replay 操作恢复。

¹²视频画面时间区间: 01:01:10–01:01:15。

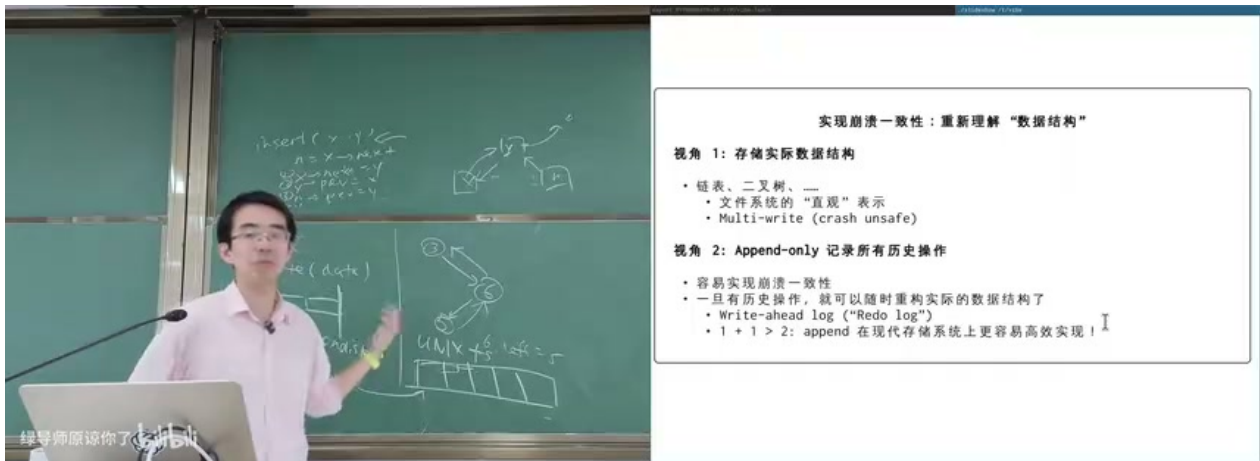


图 13: 日志区域以 append-only 方式记录操作或块写入；崩溃后可以 replay 日志恢复主数据结构。¹³

这就是 write-ahead log，也可以叫 redo log。它的原则是：在修改主数据结构前，先把足以重做这次修改的信息写到日志里，并确保日志持久化。只要日志在，主结构中间坏掉也可以通过重放日志修复。

4.2 文件系统的事务日志

文件系统不必记录每一次内存写。更实际的做法是把一次或多次系统调用涉及的块重新组成 transaction。日志中写入 transaction begin、若干块更新内容、flush、transaction end。重启恢复时，只有看到完整事务结束标记的日志才 replay。

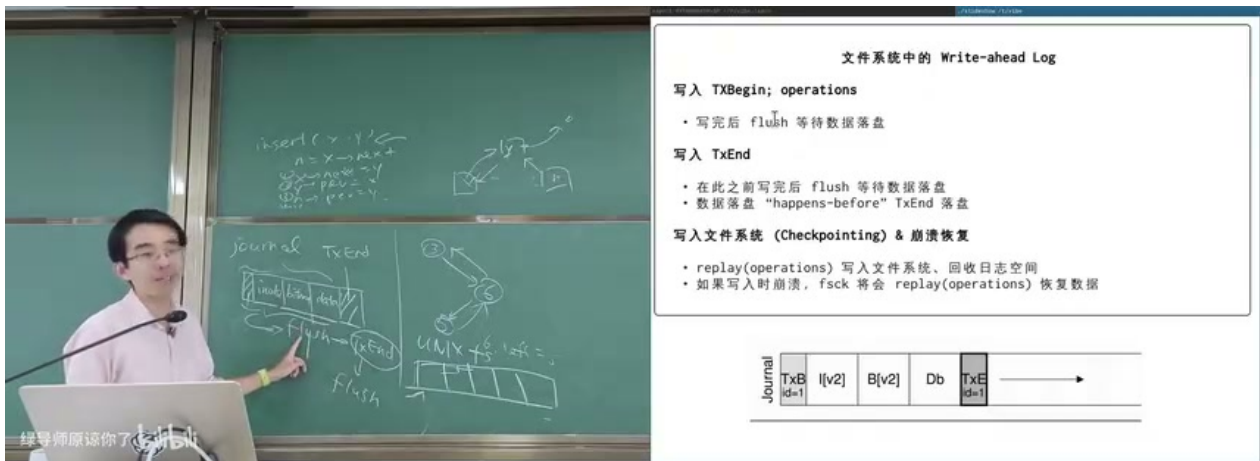


图 14: 事务日志依赖顺序关系：先写 transaction begin 和数据，flush 确认落盘，再写 transaction end；恢复时只重放完整事务。¹⁴

这里的关键不是“写了一个 end 标记”，而是 happens-before：

¹³视频画面时间区间：01:06:25–01:06:30。

¹⁴视频画面时间区间：01:10:25–01:10:30。

journal data writes → flush completes → transaction end visible

符号说明：箭头表示持久化顺序；如果恢复程序能看到 transaction end，就能推断前面的日志数据已经落盘，因此可以安全 replay。checkpointing 则是在主数据结构被更新并落盘后，把相应日志空间回收。

4.3 LSM tree 的旁支启发

课程短暂提到 LSM tree，是为了说明日志和不可变结构在存储系统中的普遍性。LSM tree 把热数据放在内存 memtable 中，每次更新先写 WAL；memtable 满后 flush 成不可变 SSTable，再通过分层 merge 均摊写入成本。数据库、键值存储和文件系统日志都在利用同一个思想：顺序追加比随机就地更新更容易做快，也更容易做一致。

4.4 JBD、checksum 和 metadata journaling

Linux ext 系列文件系统的日志由 journaling block device 支撑。为了性能，系统会合并多个系统调用到一个事务，减少 flush 次数；代价是崩溃时可能丢失更长时间窗口内的最近写入。

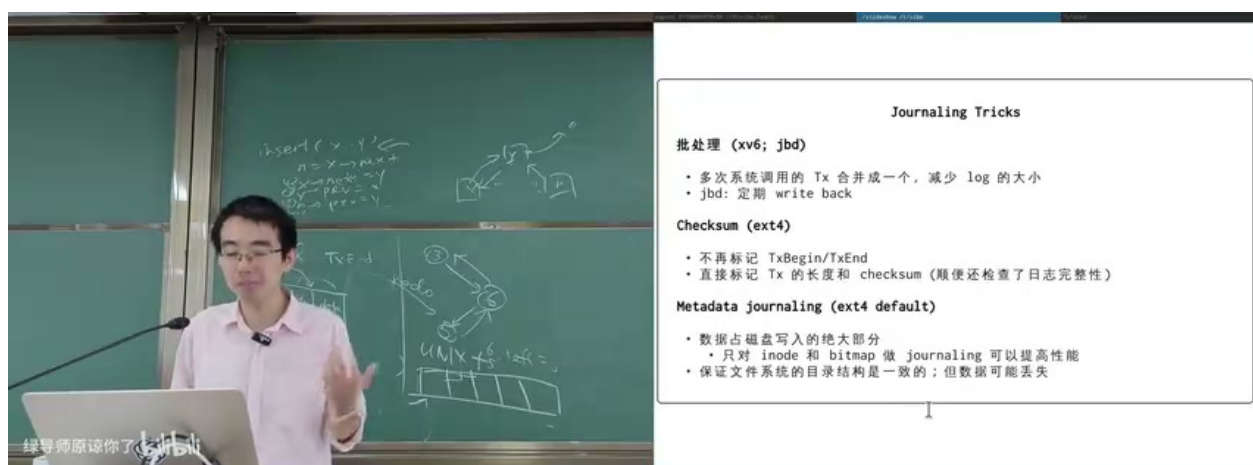


图 15: ext4/JBD 的工程技巧：合并事务、使用 checksum 判断日志完整性，并默认采用 metadata journaling。¹⁵

ext4 还可以用 checksum 和长度字段减少同步开销。恢复时读取日志记录，如果长度和 checksum 匹配，就认为该记录完整并 replay；如果不匹配，就丢弃后续尾部。这把“是否完整落盘”的判断从强制等待，部分转化为可验证的数据格式。

¹⁵视频画面时间区间：01:14:05–01:14:10。

metadata journaling 的语义

默认模式下，ext4 主要 journal inode、bitmap、目录项等元数据，而不把普通文件数据全部写入日志。这样能保证目录树结构一致，避免 inode 丢失、bitmap 自相矛盾、目录图损坏；但最近写入的文件内容仍可能丢失或变成旧内容。应用程序如果需要自己的数据持久化语义，必须显式使用同步 API。

4.5 本章小结

日志把“就地更新多个块”的危险问题转化成“顺序追加完整记录”的可控问题。事务 begin/end、flush、checksum、checkpointing 和批量提交共同构成日志文件系统的工程核心。文件系统可以保证自身元数据一致，但它不自动保证应用层每一次保存操作都符合用户直觉。

5 应用程序也要面对崩溃一致性

很多程序员以为文件系统有日志，保存文件就自然安全。课程最后强调：这是一种危险误解。metadata journaling 保证的是文件系统元数据结构，不是任意应用更新协议。

5.1 错误的保存文件方式

最糟糕的保存方式是用 `O_TRUNC` 直接打开原文件再写入。文件在 `open` 时已经被截断；如果程序随后崩溃或磁盘写满，旧内容已经没了，新内容也未必完整。

更常见但仍有问题的方式是：

```
1 fd = open("a.txt.tmp", O_CREAT | O_WRONLY, 0666);
2 write(fd, new_content, len);
3 close(fd);
4 unlink("a.txt");
5 rename("a.txt.tmp", "a.txt");
```

Listing 2: 看似安全但仍可能丢数据的保存流程

在 metadata journaling 下，创建临时文件、删除旧文件、重命名都是元数据操作，会进入日志；但临时文件的数据内容可能还没落盘。崩溃后系统可能 replay 出“创建了一个空 tmp 文件、删除了旧 a.txt”，最终用户文件消失。

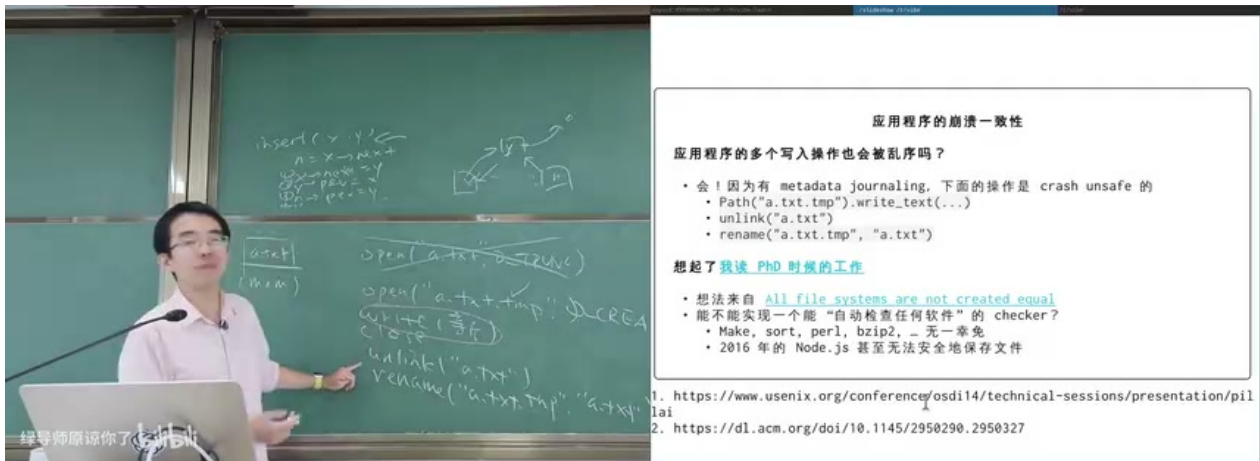


图 16: 保存文件的直觉流程在 metadata journaling 下仍可能 crash unsafe: 元数据持久化不等于文件内容持久化。¹⁶

5.2 sync 系列系统调用

应用要建立自己的持久化顺序，需要调用同步 API:

- `sync()`: 要求系统把所有脏数据尽量写回，范围最大，代价也大。
- `syncfs(fd)`: 同步某个文件描述符所在的整个文件系统。
- `fsync(fd)`: 同步指定文件的数据和必要元数据。
- `fdatasync(fd)`: 同步指定文件的数据，以及为了读取这些数据必须持久化的元数据。

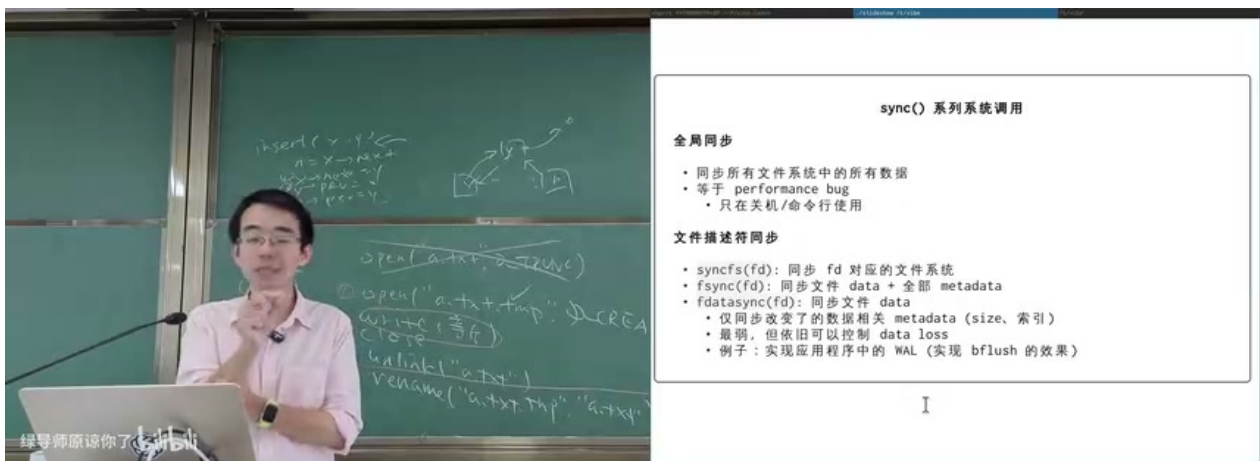


图 17: sync 系列 API 让应用显式要求落盘；数据库常偏好 `fdatasync` 以避免不必要 inode 更新。¹⁷

对于预分配固定大小的数据文件，写入某个偏移不会改变文件大小和块映射；这时 `fdatasync` 可以只保证数据落盘，避免额外 inode 元数据写入，常用于数据库 redo log。

¹⁶ 视频画面时间区间：01:16:50–01:16:55。

¹⁷ 视频画面时间区间：01:20:15–01:20:20。

5.3 安全保存：临时文件、同步、原子 rename

更合理的保存文件协议是：

```
1 fd = open("a.txt.tmp", O_CREAT | O_WRONLY | O_TRUNC, 0666);
2 write_all(fd, new_content, len);
3 fsync(fd);           // 或 fdatasync(fd), 先保证新内容落盘
4 close(fd);
5 rename("a.txt.tmp", "a.txt"); // 原子替换
6 fsync(dirfd);        // 严格场景下还要同步父目录项
```

Listing 3: 应用层 crash-safe 保存文件的核心协议

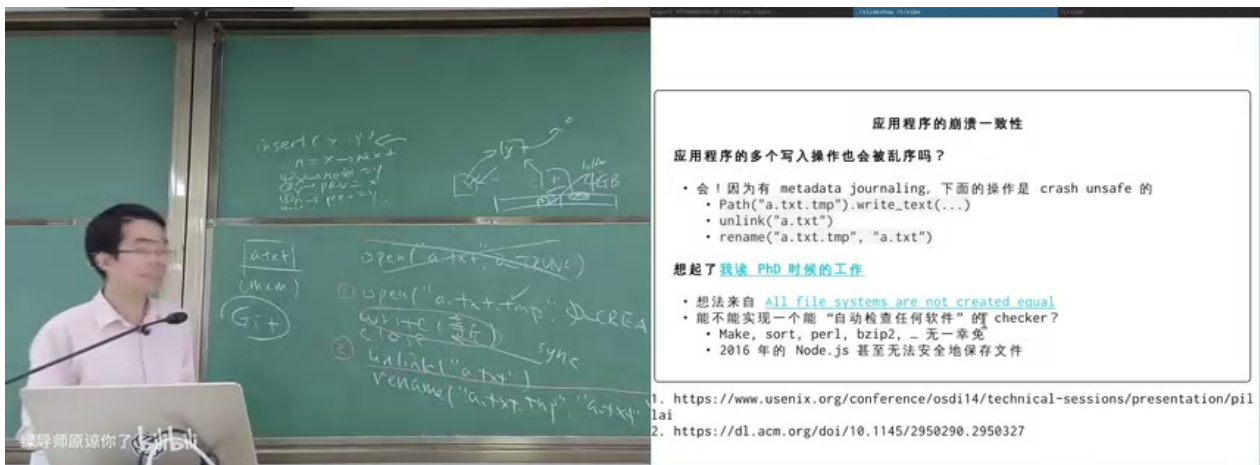


图 18: `rename` 提供原子替换语义：要么旧文件仍在，要么新名字已经指向新文件；配合同步可构造应用层一致性。¹⁸

`rename` 的价值在于它是命名空间中的原子操作。如果新文件数据已经落盘，最后一步原子替换可以让用户看到“旧版本或新版本”，而不是半个版本。许多历史程序曾经在这个问题上出错，包括 in-place sort、某些构建工具、脚本语言的原地替换选项，以及早期缺少安全保存 API 的 Electron/NodeJS 生态。

5.4 从论文到可复现实验

课程结尾用一个 crash consistency checker 说明系统研究的新工作方式：用系统调用 trace 建模程序保存文件的过程，枚举可能崩溃点和落盘子集，再让工具判断是否会造成文件丢失或错误版本。

¹⁸视频画面时间区间：01:22:25–01:22:30。

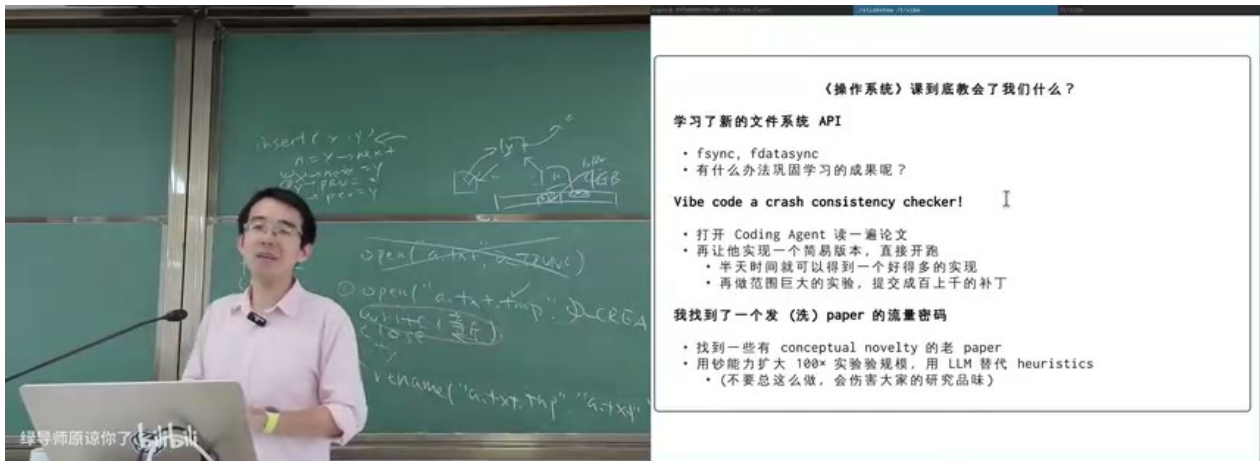


图 19: 课程结尾把 `fsync/fdatasync`、crash consistency checker 和 AI 辅助实验联系起来：系统问题可以被建模、枚举和复现。¹⁹

这也是操作系统课的重要训练：遇到复杂软件，不要停在抽象 API 的表面。可以写 trace、读规范、构造镜像、枚举故障、复现实验。文件系统的一致性不是“背几个 API”，而是把程序、内核、设备和崩溃模型一起纳入推理。

5.5 本章小结

文件系统日志只保证文件系统自己的元数据结构不坏。应用程序若要保证“保存成功后数据真的在”，必须自己建立写入、同步、重命名之间的持久化顺序。安全保存文件的核心是先写临时文件并同步内容，再用原子 `rename` 替换名字，必要时同步父目录。数据库、版本控制系统和编辑器都必须认真面对这套协议。

6 总结与延伸

本讲的主线可以压缩成一句话：文件系统是在块设备上实现的持久化数据结构，而 crash consistency 是这个数据结构区别于普通内存数据结构的难点。

6.1 机制总结

四层心智模型

第一层，块设备提供块数组和同步屏障。第二层，文件系统用 super block、FAT、inode、bitmap、目录项等结构表达目录树和文件内容。第三层，日志用 append-only 和 replay 保证元数据结构在崩溃后可恢复。第四层，应用程序用 `fsync/fdatasync/rename` 构造自己的持久化协议。

¹⁹视频画面时间区间：01:26:15–01:26:20。

FAT 和 UNIX inode 告诉我们，文件系统设计必须从 workload 出发：小文件多，大文件少；读文件头多，顺序读完整大文件少；目录项通常不极端巨大；缓存能显著改变数据布局的性能意义。Journaling 告诉我们，持久化顺序比代码顺序更重要；没有 flush 或可验证完整性的 checksum，恢复程序无法知道哪些写入真的发生。

6.2 常见误区

三个容易混淆的保证

原子性不是持久性：`rename` 可以保证名字切换原子，但不保证新文件数据已经落盘。元数据一致性不是应用数据一致性：`metadata journaling` 可以保护目录树，却不保护普通文件内容。系统调用返回不是设备持久化：`write` 返回通常只说明数据进入内核缓存，不说明已经到达非易失介质。

6.3 给实验和工程的建议

做文件系统实验时，应该把镜像看成普通二进制数据结构：先定位 super block，再定位位图、inode 或 FAT，再写遍历器和一致性检查器。做应用工程时，应该把保存文件当作协议设计：明确哪些状态允许崩溃后出现，哪些状态绝不能出现，并用同步 API 和原子操作建立顺序。

6.4 拓展阅读

建议课后阅读三类材料：FAT 文件系统规范，用来训练“从手册到代码”的能力；xv6 文件系统日志实现，用来看到最小可理解 journaling；ext4/JBD 文档或 Linux 源码注释，用来理解工业系统如何在性能、批量提交、checksum 和恢复速度之间折中。

6.5 最终 takeaways

- 文件系统不是黑箱，它是可以被读、被画、被扫描、被恢复的数据结构。
- 块设备写入可以乱序完成，崩溃会留下任意落盘子集，因此多块更新天然 crash unsafe。
- 日志的本质是先记录可重放操作，再修改主结构；恢复时只相信完整事务。
- metadata journaling 保护文件系统结构，不自动保护应用数据语义。
- 真正可靠的程序需要显式考虑 `write`、`fsync/fdatasync`、`rename` 和目录同步之间的顺序。